유한체와 다항식

halo2는 유한체(Finite Field) 위에서 정의된 다항식을 사용하여 영지식 증명을 구현합니다. 이 문서에서는 halo2에서 사용되는 수학적 기초 개념들을 설명합니다.

유한체 (Finite Field)

유한체란?

유한체는 유한한 개수의 원소를 가진 체(Field)입니다. 체는 덧셈, 뺄셈, 곱셈, 나눗셈 연산이 정의되고 특정 공리들을 만족하는 대수적 구조입니다.

halo2에서는 주로 **소수 p를 법으로 하는 유한체** $\mathbb{F}$**_p**를 사용합니다. 여기서 p는 큰 소수이며, 유한체의 원소는 {0, 1, 2, ..., p-1}입니다.

halo2에서 사용하는 유한체

halo2는 다음과 같은 유한체들을 지원합니다:

- **BN254 곡선의 스칼라 필드**: p = 21888242871839275222246405745257275088548364400416034343698204186575808495617
- **Pasta 곡선들**: Pallas와 Vesta 곡선의 스칼라 필드

```
use halo2_proofs::halo2curves::bn256::Fr as Bn256Fr;

use halo2_proofs::halo2curves::pasta::Fp as PastaFp;


// BN254 스칼라 필드의 원소

let  a = Bn256Fr::from(42);

let  b = Bn256Fr::from(17);

let  c = a + b; // 유한체에서의 덧셈
```

```
// Pasta 필드의 원소
let x = PastaFp::from(100);
let y = PastaFp::from(200);
let z = x * y; // 유한체에서의 곱셈
```

유한체 연산의 특징

유한체에서의 연산은 일반적인 정수 연산과 다릅니다:

```
use halo2_proofs::halo2curves::bn256::Fr;

let a = Fr::from(5);
let b = Fr::from(3);

// 덧셈
let sum = a + b; // 8

// 곱셈
let product = a * b; // 15

// 나눗셈 (곱셈의 역원)
let quotient = a / b; // 5 * 3^(-1) mod p

// 거듭제곱
let power = a.pow([2]); // 25
```

다항식 (Polynomial)

다항식의 정의

halo2에서 다항식은 유한체 위에서 정의됩니다. 일반적인 형태는 다음과 같습니다:

$f(X) = a_0 + a_1X + a_2X^2 + \ldots + a_nX^n$

여기서 계수 $a_0, a_1, \ldots, a_n$은 모두 유한체의 원소입니다.

halo2에서의 다항식 활용

halo2는 다항식을 다음과 같은 용도로 사용합니다:

1. **회로 표현**: 산술 회로를 다항식으로 인코딩
2. **제약조건**: 회로의 정확성을 다항식 등식으로 표현
3. **증명 생성**: 다항식 commitment를 통한 증명

```
// 다항식 예제 (의사코드)
// f(X) = 3X² + 2X + 1 을 표현
let coefficients = vec![Fr::from(1), Fr::from(2), Fr::from(3)]; // [상수항, X계수, X²계수


// 특정 점에서의 다항식 값 계산
fn evaluate_polynomial(coeffs: &[Fr], x: Fr) -> Fr {
let mut result = Fr::zero();
let mut power = Fr::one();
for coeff in coeffs {
result += *coeff * power;
power *= x;
}
result
```

```
}


let  x = Fr::from(5);

let  value = evaluate_polynomial(&coefficients, x); // f(5) = 3*25 + 2*5 + 1 = 86
```

Polynomial Commitment

Commitment란?

Polynomial Commitment는 다항식에 대한 "약속"을 의미합니다. 증명자가 특정 다항식을 알고 있다는 것을 검증자에게 증명하면서도, 다항식 자체는 공개하지 않는 암호학적 기법입니다.

KZG Commitment

halo2는 KZG (Kate-Zaverucha-Goldberg) polynomial commitment scheme을 사용합니다:

1. **Setup**: 신뢰할 수 있는 설정을 통해 공개 매개변수 생성
2. **Commit**: 다항식 f(X)에 대한 commitment C = $[f(\tau)]_1$ 생성
3. **Open**: 특정 점 z에서 f(z) = y임을 증명
4. **Verify**: commitment와 opening proof를 검증

```
// KZG commitment 예제 (의사코드)

use halo2_proofs::poly::kzg::commitment::{KZGCommitmentScheme, ParamsKZG};


// 1. Setup - 신뢰할 수 있는 설정

let  params = ParamsKZG::<Bn256>::setup(degree, &mut  rng);


// 2. Commit - 다항식에 대한 commitment 생성
```

```
let polynomial = // ... 다항식 정의
let commitment = params.commit(&polynomial);

// 3. Open - 특정 점에서의 값과 증명 생성
let point = Fr::from(42);
let value = evaluate_polynomial(&polynomial, point);
let proof = params.open(&polynomial, point);

// 4. Verify - commitment와 proof 검증
let is_valid = params.verify(&commitment, point, value, &proof);
```

halo2에서의 실제 활용

1. 회로 다항식

halo2에서 회로는 여러 다항식으로 표현됩니다:

- **Gate 다항식**: 각 gate의 제약조건을 표현
- **Copy 다항식**: 셀 간의 등식 제약조건을 표현
- **Lookup 다항식**: lookup table과의 관계를 표현

```
// Gate 다항식 예제
// 덧셈 gate: a + b = c
// 다항식으로 표현: a(X) + b(X) - c(X) = 0

fn addition_gate_polynomial(
a_poly: &Polynomial<Fr>,
```

```
b_poly: &Polynomial<Fr>,
c_poly: &Polynomial<Fr>
) -> Polynomial<Fr> {
a_poly + b_poly - c_poly
}
```

2. 제약조건 검증

모든 제약조건이 만족되면 해당 다항식들이 0이 됩니다:

```
// 제약조건 다항식이 모든 점에서 0인지 확인
fn check_constraints(constraint_polys: &[Polynomial<Fr>]) -> bool {
for poly in constraint_polys {
for i in 0..poly.len() {
if poly.evaluate_at_point(Fr::from(i as u64)) != Fr::zero() {
return false;
}
}
}
true
}
```

3. 증명 생성 과정

```
// 증명 생성 과정 (단순화된 버전)
fn generate_proof(
circuit: &Circuit,
```

```
witness: &Witness,
params: &ParamsKZG<Bn256>
) -> Proof {
// 1. 회로를 다항식으로 변환
let constraint_polys = circuit.to_polynomials(&witness);
// 2. 각 다항식에 대한 commitment 생성
let commitments: Vec<_> = constraint_polys
.iter()
.map(|poly| params.commit(poly))
.collect();
// 3. 랜덤 challenge 생성
let challenge = generate_challenge(&commitments);
// 4. Opening proof 생성
let opening_proofs: Vec<_> = constraint_polys
.iter()
.map(|poly| params.open(poly, challenge))
.collect();
Proof {
commitments,
opening_proofs,
// ... 기타 필드들
}
}
```

성능 고려사항

1. 다항식 차수

다항식의 차수가 높을수록 더 많은 계산이 필요합니다:

```
// 차수에 따른 성능 영향
let low_degree = 10; // 빠른 연산
let high_degree = 1000; // 느린 연산

// Setup 시간도 차수에 비례
let params_small = ParamsKZG::<Bn256>::setup(low_degree, &mut rng);
let params_large = ParamsKZG::<Bn256>::setup(high_degree, &mut rng);
```

2. 메모리 사용량

큰 다항식은 많은 메모리를 사용합니다:

```
// 메모리 효율적인 다항식 처리
fn process_large_polynomial_efficiently(poly: &Polynomial<Fr>) {
// 청크 단위로 처리
const CHUNK_SIZE: usize = 1024;
for chunk in poly.coefficients().chunks(CHUNK_SIZE) {
// 청크별로 처리
process_chunk(chunk);
}
}
```

다음 단계

유한체와 다항식의 기본 개념을 이해했다면, 다음 문서들을 읽어보세요:

- 회로와 제약조건: 이 수학적 개념들이 어떻게 회로로 구현되는지 학습
- 증명과 검증: 다항식 commitment가 실제 증명 시스템에서 어떻게 사용되는지 이해

참고 자료

- KZG Polynomial Commitment 논문
- halo2 curve 라이브러리 문서
- 유한체 이론 기초